

Livrable 4: Service à la demande de PC sur mesure

SEG 2505 - Introduction au génie logiciel

Automne 2024

School of Electrical Engineering and Computer Science

University of Ottawa

Course Coordinator: Frerebeau Laurent

Référence sur le projet GitHub:

Lien GitHub: <https://github.com/uOttawa-2024-2025-seg2505-projet/groupe-8>

Le dépôt contient :

- Les codes source Android Studio.
- Les fichiers APK pour chaque livrable.
- Les diagrammes UML (activités, classes, séquences, états).
- Les fichiers README et les tags des livrables.

Choix de conception:

Description des choix de conception majeurs :

Architecture logicielle :

Architecture modulaire : Pour séparer clairement les rôles et responsabilités (Requester, StoreKeeper, Assembler), une architecture modulaire a été adoptée. Cela permet une meilleure maintenabilité et évolutivité de l'application en isolant les différentes fonctionnalités dans des modules distincts.

Gestion de la base de données :

Firebase : Au lieu de SQLite, Firebase a été choisi comme solution de base de données.

Firebase offre plusieurs avantages tels que :

Base de données en temps réel : Permet la synchronisation instantanée des données entre les clients et le serveur, facilitant ainsi la gestion des commandes et des stocks en temps réel.

Authentification sécurisée : Firebase Authentication fournit des méthodes robustes pour gérer l'inscription et l'authentification des utilisateurs, renforçant ainsi la sécurité du système.

Scalabilité : Firebase est conçu pour s'adapter facilement à une augmentation du nombre d'utilisateurs et des volumes de données sans nécessiter de modifications majeures de l'architecture.

Intégration facile avec Android : Firebase s'intègre de manière transparente avec Android Studio, simplifiant le développement et la gestion des données.

Interface utilisateur (UI) :

Respect des directives Android : L'interface utilisateur a été conçue en suivant les directives de conception d'Android pour assurer une expérience utilisateur cohérente, intuitive et agréable. L'application est disponible en français et en anglais pour répondre à une audience plus large.

Gestion des rôles :

Contrôle d'accès basé sur les rôles : L'application implémente des restrictions d'accès basées sur les rôles (Requester, StoreKeeper, Assembler, Administrator) pour sécuriser et organiser les fonctionnalités accessibles à chaque type d'utilisateur. Firebase Authentication facilite la gestion des rôles et des permissions.

Conformité au scénario :

Cycle de commande complet : Le scénario principal de l'application suit le cycle de commande, depuis la création jusqu'à la livraison, en passant par la gestion des stocks et l'assemblage des PC sur mesure. Firebase permet de suivre et de gérer efficacement chaque étape de ce cycle grâce à ses capacités de gestion des données en temps réel.

Justifications de l'utilisation de Firebase :

Facilité de développement : Firebase simplifie de nombreuses tâches backend, telles que l'authentification, la gestion de la base de données et l'hébergement, ce qui permet de se concentrer davantage sur le développement des fonctionnalités frontend.

Temps réel et réactivité : La capacité de Firebase à synchroniser les données en temps réel est cruciale pour une application où les informations sur les commandes et les stocks doivent être constamment mises à jour et accessibles instantanément par les utilisateurs.

Sécurité : Firebase offre des règles de sécurité granulaires qui permettent de contrôler l'accès aux données en fonction des rôles des utilisateurs, assurant ainsi que seules les personnes autorisées peuvent effectuer certaines actions ou accéder à des informations spécifiques.

Scalabilité et maintenance : Firebase est une solution gérée qui évolue avec les besoins de l'application, réduisant ainsi les efforts nécessaires pour la maintenance et l'évolution de l'infrastructure backend.

Login and Signup:


uOttawa

Uottawa Hardware Sign up

It's quick and easy! Uottawa Hardware is about computing tailored to you.

First Name

Last Name

First Name

Last Name

Email Address

Email Address

✓

Username

Username

Password

Password

👁

Confirm Password

Confirm Password

SIGN UP


uOttawa

Uottawa Hardware

Please login with your information

Email

✓

Password

👁

☐ Remember me

Forgot password

LOGIN

or

SIGN UP

Admin: Menu principal et navigation:


PC SUR MESURE
VOTRE PC. VOTRE STYLE

Admin Dashboard

MANAGE STOCK

ADD USERS

REMOVE USERS

CREATE ORDER

MY ORDERS

LOGOUT

Search Users

Item 1
Sub Item 1

Item 2
Sub Item 2

Item 3
Sub Item 3

Item 4
Sub Item 4

Item 5
Sub Item 5

Item 6
Sub Item 6

Item 7
Sub Item 7

Item 8
Sub Item 8

Item 9
Sub Item 9

Item 10
Sub Item 10

Admin Dashboard

Delete Database
Delete Requesters, Stock, Commands

DELETE

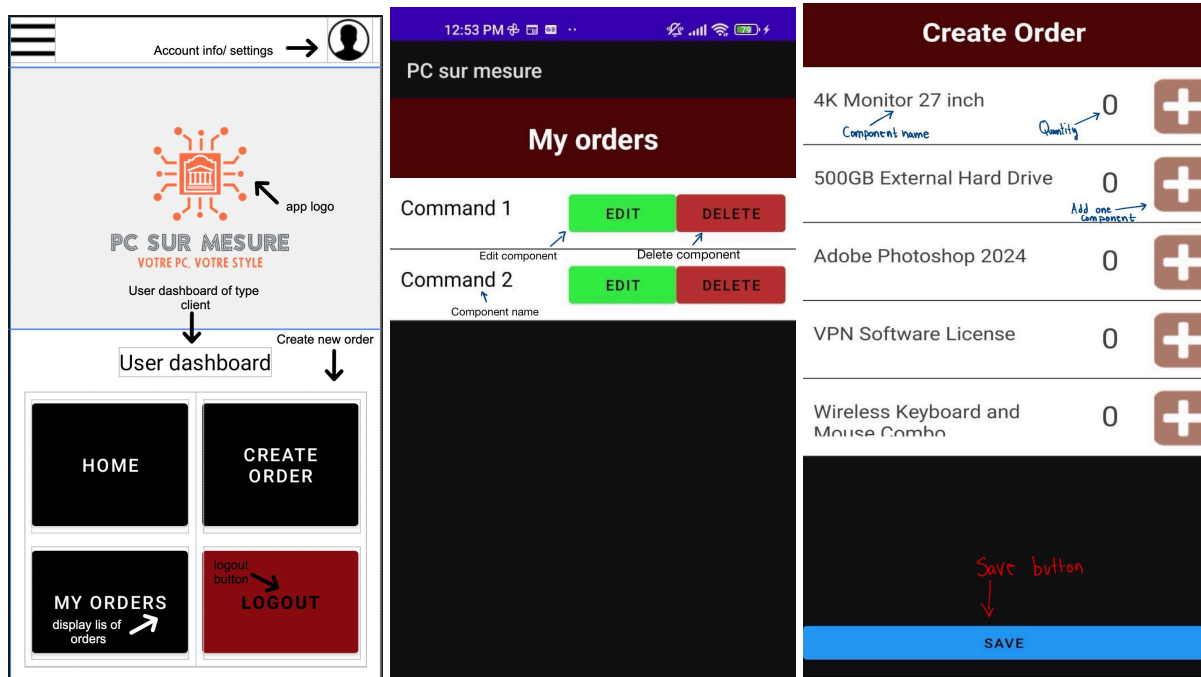
Reinitialize database
Override all Requesters and Stock

✎

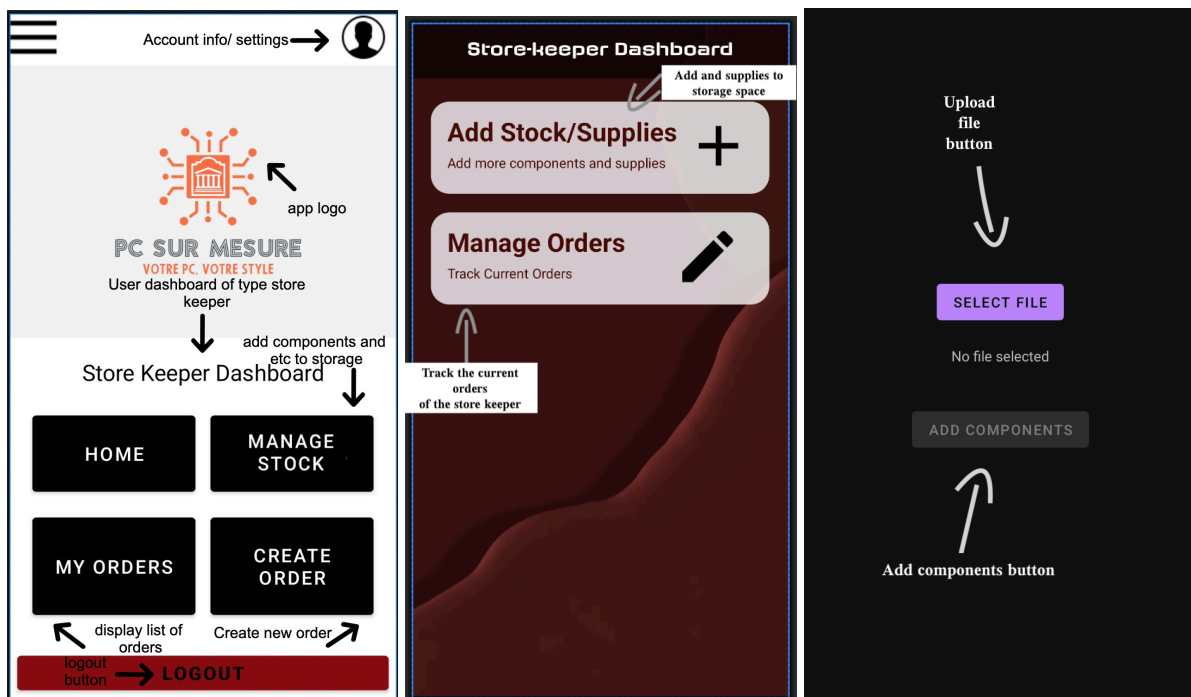
Reinitialize Stock
Override Stock

+

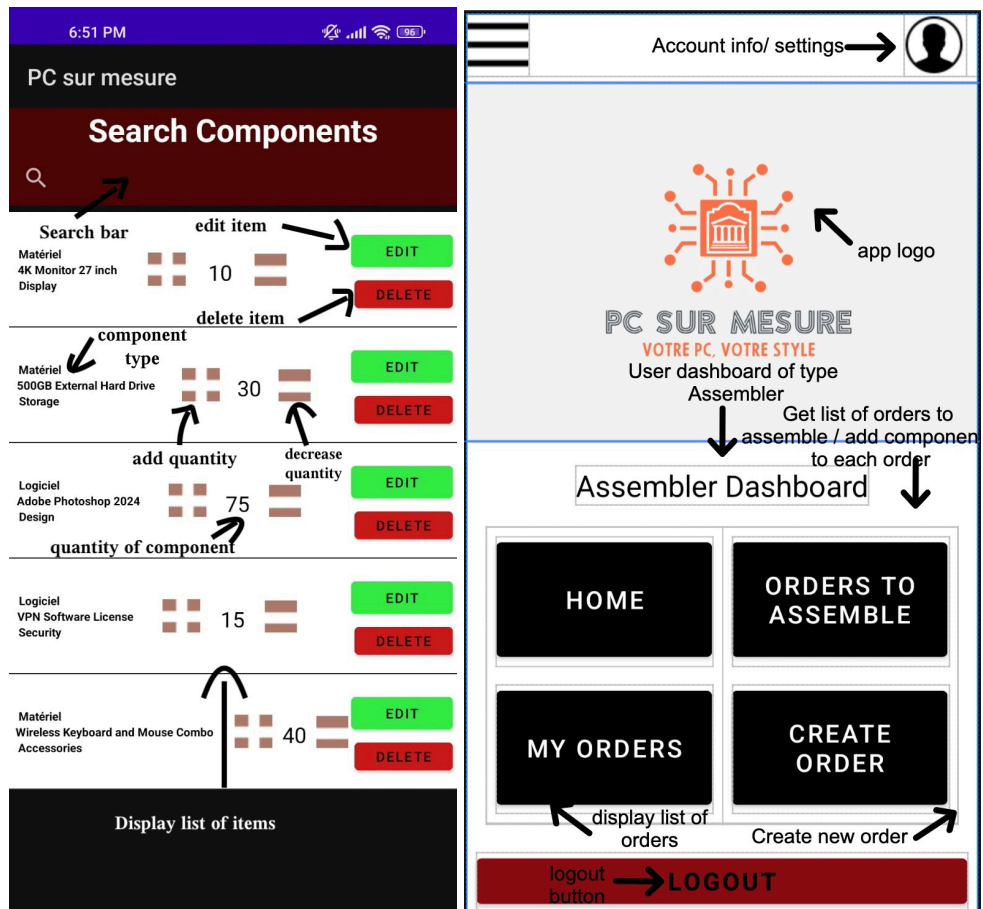
Client: Menu principal et navigation:



Storekeeper: Menu principal et navigation:



Assembler: Menu principal et navigation:



Description des exigences supplémentaires proposées ou traitées

Exigences supplémentaires proposées et traitées

Fonctionnalité de recherche avancée dans le stock et les commandes

Description : Ajout d'une barre de recherche permettant aux utilisateurs de filtrer :

Les composants du stock par type ou sous-type.

Les commandes par état ou date de création.

Réalisation : Utilisation de Firebase Firestore pour effectuer des requêtes complexes. La fonctionnalité offre un gain de temps aux utilisateurs, notamment au StoreKeeper pour gérer efficacement un grand inventaire.

Impact :

Améliore la productivité des utilisateurs.

Renforce la facilité d'utilisation de l'application.

Historique des actions utilisateur

Description : Mise en place d'un système de journalisation pour enregistrer les actions importantes réalisées par les utilisateurs (ex. : création de commandes, modifications dans le stock).

Réalisation : Les logs sont stockés dans Firebase pour permettre un suivi des activités, utile pour l'administrateur en cas de vérifications ou de résolution de problèmes.

Impact :

Assure une meilleure traçabilité.

Permet d'identifier et de résoudre des erreurs ou des abus potentiels.

Exigences supplémentaires écartées

Système de chat intégré entre les rôles

Description : Proposition d'un système de messagerie directe permettant des échanges entre les différents utilisateurs du projet.

Raison de l'abandon :

Complexité technique et dépassement du périmètre initial du projet.

Existence d'outils de communication externes (email, Slack, etc.) déjà utilisés par les utilisateurs.

Impact :

La fonctionnalité a été jugée non essentielle pour le cycle de commande et d'assemblage.

Ajout d'une option de personnalisation avancée pour les PC

Description : Proposition d'une interface permettant aux Requesters de configurer des spécifications détaillées, comme les performances spécifiques des composants.

Raison de l'abandon :

Risque de complexité excessive pour la gestion du stock et des commandes.

Nécessite des mises à jour fréquentes des options dans l'application pour refléter les stocks disponibles.

Impact :

L'approche actuelle (choix parmi des listes prédéfinies) reste simple et efficace.

Résumé et impact global

Ces exigences supplémentaires, bien que limitées en nombre, ont été soigneusement sélectionnées pour maximiser la valeur ajoutée tout en respectant les contraintes de temps et de ressources. Les fonctionnalités mises en œuvre (recherche avancée, historique) renforcent l'efficacité et l'interactivité de l'application, tandis que les fonctionnalités écartées ont permis de concentrer les efforts sur les éléments essentiels.

Retour d'expérience

Organisation de l'équipe

Dès le début du projet, nous avons pris le temps de structurer l'équipe et d'établir des méthodes efficaces pour respecter les délais tout en maintenant la qualité du produit.

Répartition des rôles : Nous avons réparti les rôles en fonction des compétences et des intérêts de chaque membre de l'équipe :

Responsable Requester : Gestion du système de commandes.

Responsable StoreKeeper : Gestion du stock.

Responsable Assembler : Gestion des étapes d'assemblage et des commandes livrées.

Responsable Administrateur : Gestion des utilisateurs et réinitialisation de la base de données.

Coordinateur technique : Supervision des intégrations et révisions du code.

Chaque membre a également contribué aux tâches transversales, telles que la conception UML, les tests et la rédaction de documentation.

Outils de collaboration :

GitHub : Utilisé pour le contrôle de version, les revues de code et l'organisation des tâches à l'aide de GitHub Projects.

Discord : Réunions hebdomadaires pour discuter des progrès et résoudre les obstacles.

Planification des livrables :

Nous avons divisé le projet en quatre grandes étapes (conformément aux livrables) et défini des jalons intermédiaires pour chaque livrable.

Une attention particulière a été accordée à la gestion des délais, avec des "mini-délais" fixés avant les dates officielles pour permettre une période de révision.

Difficultés rencontrées et solutions apportées

Problème : Gestion des conflits sur GitHub

Description : Des conflits fréquents sont apparus lors des fusions, principalement lorsque plusieurs membres travaillaient sur les mêmes fichiers.

Solution : Nous avons adopté une convention stricte de création de branches nommées par fonctionnalité, avec des revues de code systématiques avant les fusions. Nous avons également planifié les intégrations en binômes pour minimiser les conflits.

Problème : Problèmes d'intégration Firebase

Description : Lors de la mise en place de Firebase pour la gestion des données, des erreurs liées à l'authentification et à la synchronisation des collections sont survenues.

Solution : Consultation de la documentation Firebase et des forums communautaires, en plus de tests unitaires ciblés pour valider les flux de données. Un membre a été désigné comme référent pour approfondir les fonctionnalités avancées de Firebase.

Problème : Difficulté à coordonner les rôles

Description : Certains rôles étaient interdépendants (par exemple, l'Assembleur dépendait des données du StoreKeeper), ce qui a parfois entraîné des blocages.

Solution : Nous avons organisé des sessions de "pair programming" pour résoudre rapidement les dépendances et assurer une compréhension commune des attentes.

Problème : Gestion du temps

Description : Le projet demandait une charge de travail importante, et certains membres avaient des obligations académiques ou personnelles qui compliquaient la coordination.

Solution : Adoption d'un calendrier partagé et identification des périodes où des ressources supplémentaires étaient nécessaires pour compenser les indisponibilités.

Leçons tirées pour les futurs projets

Ce qui s'est bien passé :

- Collaboration efficace : L'utilisation des outils collaboratifs (GitHub) a permis de maintenir une communication fluide et d'assurer une traçabilité des tâches.
- Planification initiale solide : La définition des rôles et des jalons dès le début a permis de réduire les imprévus et de garder une vue d'ensemble sur le projet.
- Résolution proactive des problèmes : Les réunions hebdomadaires nous ont aidés à anticiper et résoudre rapidement les obstacles techniques ou organisationnels.
- Innovation et créativité : L'ajout de fonctionnalités supplémentaires, comme les fonctionnalités de recherche avancée dans le stock et les commandes, a enrichi l'application et montré notre capacité à dépasser les attentes initiales.

Ce qui aurait pu être mieux :

- Gestion des dépendances : Mieux anticiper les interdépendances entre les rôles pour éviter les blocages.
- Amélioration des tests : Les tests unitaires auraient pu être plus nombreux et plus tôt dans le processus, ce qui aurait réduit les régressions en fin de projet.
- Engagement uniforme : La contribution de certains membres a été moins constante, ce qui a entraîné une charge de travail inégale. Une meilleure planification des tâches individuelles aurait permis de répartir équitablement les efforts.

Idées pour les projets futurs (Brainstorming d'équipe)

- Formaliser une documentation technique dès le début : Un guide clair pour les conventions de code et l'utilisation des outils aurait accéléré l'intégration des membres.
- Utiliser un gestionnaire de tâches centralisé (ex. : Jira) : Pour une gestion encore plus précise des responsabilités et des délais.
- Tester dès les premières étapes : Introduire des tests unitaires et d'intégration dès le premier livrable pour détecter rapidement les anomalies.
- Organiser des sessions de brainstorming en présentiel : La collaboration en face à face pourrait renforcer l'esprit d'équipe et réduire les malentendus.

	Livable 1	Livable 2	Livable 3	Livable 4
Sofia El Ouazzani [300367668]	ui accountsettings ui signup ui login ui homepages rapport	Rapport ui storekeeper	Readme File Front Ui mesCommandes	Rapport Rapport finale
Youssef Cherkaoui [300360743]	ui homepages Drawer menu functionality Front end editUser method	Backend Frontend	Backend Frontend	Read me Frontend
Yasser El Mouatadir	-Set up firebase -Authentification -Add/removeUser methods	-Readme	-Tests	-Backend -Tests
Amine Baba [300346890]	-UML -ReadMe -UI Adduser -UI remove user List	-3 UIs: ItemComponents Edit_remove_added_components	-3UIs: Creation_order Edit_order My_orders (et leurs listes)	2UIs: list orders waitlist list components of command
Ismail El Khayati	ui homepages ui remove user list ui sign up getAllusers	Frontend	Tests	Tests